

FRAMEWORK VÀ LỘ TRÌNH



TÓM TẮT

Hai chương khép cây. Chương 23 là bản đồ công cụ — ai đã cài sẵn cái gì, và quan trọng hơn, *đọc mã nguồn nào để học được gì*; nó được xếp cuối vì nó hết hạn nhanh nhất và vì nó chỉ có nghĩa khi bạn đã biết mình đang tìm gì. Chương 24 là lộ trình năm giai đoạn: thứ tự học, với lý do cho từng thứ tự, và một lập luận rằng giai đoạn bốn — tự dựng dữ liệu và đo thất bại — là giai đoạn bị bỏ qua nhiều nhất và có giá trị cao nhất. Nếu chỉ nhớ một điều: *đọc mã nguồn của một hệ trường thành dạy nhiều hơn đọc bài báo của nó*, vì phần lớn kiến thức ở Phần C và Phần D không tồn tại trong bài báo.

Phần này ngắn và có vai trò khác hẳn bốn phần trước. Bốn phần trước xây dựng hiểu biết; phần này là điều hướng — nó nói bạn nên đi đâu tiếp và theo thứ tự nào.

Một lưu ý về tính hết hạn. Chương 23 chốt tại tháng 9/2026 và nó là phần hết hạn nhanh nhất của cả cây cùng với C/16 và C/17. Các dự án mã nguồn mở đối, các thư viện đối API, và các hệ thống mới xuất hiện. Điều *không* đổi nhanh là *nguyên tắc chọn* — đọc cái gì để học cái gì — nên chương 23 được viết xoay quanh nguyên tắc ấy chứ không xoay quanh danh sách.

Chương 24 thì ngược lại: nó gần như không hết hạn, vì nó nói về thứ tự học, và thứ tự ấy suy ra từ cấu trúc phụ thuộc của chính lĩnh vực. Lie group phải đến trước bundle adjustment vì BA cần Jacobian trên đa tạp; observability phải đến sau BA vì nó là câu hỏi về ma trận của BA. Những quan hệ ấy không đổi.

Bảng 1. Bản đồ Phần E: hai chương khép cây.

Ch.	Thư mục	Nội dung	Mức
23	23-framework-va-cor	Nguyên tắc chọn mã nguồn để đọc; sáu hệ phải đọc và học được gì từ mỗi hệ; thư viện nền; bộ dữ liệu và công cụ đánh giá; chốt 9/2026	•
24	24-lo-trình	Lộ trình năm giai đoạn với lý do cho từng thứ tự; lộ trình rút gọn cho người có ít thời gian; và lập luận vì sao giai đoạn bốn quan trọng nhất	••

Chương 23 là tham khảo — đọc lướt một lần để biết có gì, rồi quay lại khi cần chọn công cụ hoặc khi muốn đọc mã nguồn một chủ đề cụ thể. Nó chốt tại 9/2026 và nên rà lại mỗi sáu tháng. Chương 24 thì nên đọc *sớm*, thậm chí trước khi đọc Phần A — vì nó nói thứ tự nên đi, và biết thứ tự trước khi bắt đầu tiết kiệm rất nhiều thời gian. Nếu bạn đang đọc cây này lần đầu và chưa xem chương 24, hãy xem nó ngay bây giờ rồi quay lại.

Framework và công cụ (Frameworks and Tools)

PHỤ THUỘC Toàn bộ cây — chương này chỉ có nghĩa khi bạn đã biết mình đang tìm gì. Đọc nó trước khi đọc Phần A là lãng phí.

TÓM TẮT

Chương này là bản đồ công cụ, và nó được viết xoay quanh một câu hỏi duy nhất: *đọc mã nguồn nào để học được gì?* Cách tổ chức ấy là có chủ ý — một danh sách dự án hết hạn trong hai năm, nhưng nguyên tắc “muốn hiểu marginalization thì đọc VINS-Mono” thì bền hơn nhiều. Nội dung gồm: nguyên tắc chọn mã nguồn để đọc, và vì sao đọc mã dạy nhiều hơn đọc bài báo với phần lớn nội dung Phần C và Phần D; sáu hệ thống đáng đọc trọn vẹn, mỗi hệ kèm chỉ dẫn *đọc phần nào trước*; các thư viện nền và tiêu chí chọn giữa chúng; và bộ dữ liệu cùng công cụ đánh giá. Nếu chỉ nhớ một điều: *phần lớn kiến thức ở Phần C và Phần D không tồn tại trong bài báo* — kiến trúc ba luồng, chính sách quản lý bản đồ, các cổng và ngưỡng — và cách duy nhất học chúng là đọc mã của một hệ đã trưởng thành.

CẠM BẮY

Chương này chót tại tháng 9/2026 và là phần hết hạn nhanh nhất của cây cùng với C/16 và C/17. Dự án đổi, API đổi, hệ mới xuất hiện. Rà lại mỗi sáu tháng. Tôi cũng **không xếp hạng chất lượng** các dự án. Lý do: “tốt hơn” phụ thuộc vào việc bạn cần gì, và một bảng xếp hạng sẽ sai ngay khi ai đó có ràng buộc khác tôi. Thay vào đó, mỗi mục trả lời câu “*đọc nó để học được gì*” — một câu hỏi có câu trả lời ổn định hơn nhiều.

1.1 Vì sao đọc mã nguồn

Có một sự bất đối xứng đáng nói giữa hai nửa của cây này.

Với Phần A và phần lớn Phần B — hình học, MAP, bundle adjustment, các paradigm back-end — *bài báo là nguồn tốt*. Nội dung là toán, nó viết ra được, và một bài báo tốt trình bày nó rõ hơn mã nguồn.

Với Phần C và Phần D thì ngược lại, và lý do có cấu trúc. Kiến trúc ba luồng ở C/13 mục 4A, chính sách chọn và loại keyframe, các cổng ở B/10 và B/12, cách hoà nhập loop closure với tracking — những thứ ấy *không viết được thành định lý*, chúng không phải đóng góp mà một bài báo khoe, và chúng chiếm phần lớn mã nguồn. Một bài báo mô tả hệ thống như thể nó chạy tuần tự; mã nguồn thật đầy hàng đợi, khoá, và cờ trạng thái.

Hệ quả thực hành: *với Phần C và Phần D, đọc mã là cách học chính, không phải cách bổ sung*. Và đọc mã của một hệ *đã trưởng thành* — đã chạy trên nhiều bộ dữ liệu, đã được nhiều người dùng, đã sửa nhiều lỗi — dạy nhiều hơn đọc mã của một hệ mới hơn nhưng chưa va chạm.

Cách đọc mã có hiệu quả, ba nguyên tắc.

Một — đọc theo câu hỏi, không đọc tuần tự. Mở mã với một câu hỏi cụ thể: “nó quyết định tạo keyframe thế nào?” Tìm chỗ ấy, đọc kỹ, rồi đóng lại. Đọc từ đầu tới cuối một hệ vài chục nghìn dòng là cách chắc chắn bỏ cuộc.

Hai — tìm các ngưỡng và hỏi chúng từ đâu ra. Mỗi hằng số trong mã là một quyết định. Phần lớn được chỉnh bằng thực nghiệm, và biết cái nào có cơ sở lý thuyết và cái nào không là kiến thức có giá trị.

Ba — so mã với bài báo của chính nó. Chỗ chúng khác nhau là chỗ thú vị nhất: đó là chỗ lý thuyết gặp thực tế, và thường là chỗ có một bài học không được viết ra.

1.2 Sáu hệ thống đáng đọc

Bảng 1.1. Sáu hệ thống và cái đáng học từ mỗi hệ. Cột cuối là chỉ dẫn đọc — phần nào trước.

Hệ thống	Đọc để học	Đọc phần nào trước
ORB-SLAM3	Kiến trúc SLAM đầy đủ nhất: ba luồng, covisibility, Atlas, quản lý bản đồ	Bắt đầu ở vòng lặp tracking; rồi chính sách tạo và loại keyframe (C/13); rồi loop closing và cách nó hoà nhập với tracking
VINS-Fusion	Cửa sổ trượt và marginalization, ngắn và dễ đọc nhất trong sáu	Bắt đầu ở chỗ dựng bài toán tối ưu; rồi marginalization (A/04 mục 5); rồi khởi tạo VI
OpenVINS	Bộ lọc chuẩn mực; hiệu chuẩn trực tuyến; có mô phỏng có chân trị	Chỗ định nghĩa vector trạng thái; rồi cờ FEJ và chỗ Jacobian được tính; rồi mô-đun mô phỏng — đây là chỗ tính NEES được (C/18)
GTSAM	Đồ thị nhân tử đúng cách; iSAM2; cách viết một loại nhân tử mới	Ví dụ về custom factor; rồi cấu trúc Bayes tree; rồi Marginals (A/06 mục 2B)
Ceres	Manifold, auto-diff, bộ giải thừa với Schur	Tài liệu về Manifold (A/03 mục 6); rồi các lựa chọn linear solver và ordering (A/04 mục 4D)
Kalibr	Toàn bộ tư duy hiệu chuẩn = batch MAP	Tài liệu quy trình thu dữ liệu trước mã — mỗi yêu cầu trong đó là một hàng của Bảng kích thích ở A/05

Một gợi ý về thứ tự nếu bạn đọc cả sáu: *VINS-Fusion* trước vì nó ngắn nhất và cấu trúc rõ; *OpenVINS* tiếp theo để thấy thế giới bộ lọc và để có mô phỏng; *ORB-SLAM3* thứ ba vì nó đầy đủ nhất và cũng phức tạp nhất — đọc nó trước khi có nền là nản. *Kalibr* đọc song song khi học C/14. *GTSAM* và *Ceres* đọc như tài liệu tham khảo khi cần, không đọc tuần tự.

1.3 Các hệ thống khác, theo chủ đề

Xếp theo chủ đề bạn muốn học chứ không theo mức độ nổi tiếng.

Direct method và mô hình quang trắc: *DSO* là cài đặt tham chiếu; *DM-VIO* thêm quán tính và dynamic marginalization. Đọc *DSO* để hiểu B/08 mục 2B và A/02 mục 5B.

Kỹ thuật số học và chất lượng phần mềm: [Basalt](#). Dạng căn bậc hai (B/09 mục 5A), spline $SE(3)$ cho hiệu chuẩn, và chất lượng kỹ thuật rất cao — đáng đọc như một ví dụ về cách viết phần mềm SLAM tử tế.

Semi-direct và tốc độ: [SVO-Pro](#). Đọc để thấy mục 2C của B/08 — phân công theo thể mạnh.

SfM và localization quy mô lớn: [COLMAP](#) cho SfM chuẩn công nghiệp; [HLoc](#) cho pipeline localization phân cấp (B/12 mục 2).

Kiến trúc lai học được: [DROID-SLAM](#) và [DPVO](#). Đọc để thấy ranh giới ở C/17 mục 3B được đặt ở đâu trong mã.

Quản lý bản đồ nhiều phiên: [maplab](#). Hiếm framework coi C/13 là chủ đề chính chứ không phải phụ.

Ngữ nghĩa trong đồ thị: [Kimera](#). Ví dụ về việc thông tin ngữ nghĩa cũng chỉ là một loại nhân tử.

Biểu diễn neural: [SpliTAM](#), [MonoGS](#). Chốt 9/2026; xem cảnh báo ở C/16 mục 4.

Fiducial: [TagSLAM](#). Liên quan `.../marker/`.

1.4 Thư viện nền

Lie group: [Sophus](#) là lựa chọn phổ biến nhất trong hệ sinh thái C++ SLAM; [manif](#) có API hiện đại hơn và Jacobian tường minh cho mọi phép toán, và nó đi kèm [1] vốn là tài liệu tham chiếu tốt nhất về quy ước. Tiêu chí chọn: nếu bạn đang học và cần hiểu quy ước rõ ràng, dùng [manif](#); nếu bạn đang tích hợp vào hệ sinh thái có sẵn, dùng cái mà hệ đó dùng.

Tối ưu: [Ceres](#) cho auto-diff và sự trưởng thành; [GTSAM](#) cho đồ thị nhân tử và smoothing gia tăng; [g2o](#) cho mã ngắn dễ đọc (tốt để học, không nhất thiết để dùng); [SymForce](#) cho việc sinh mã Jacobian tối ưu; [Theseus](#) cho tối ưu khả vi (C/17 mục 3B).

Tiêu chí chọn giữa Ceres và GTSAM, vốn là lựa chọn hay gặp nhất: nếu bài toán của bạn là một bài toán tối ưu tĩnh — hiệu chuẩn, BA ngoại tuyến — thì Ceres đơn giản hơn. Nếu nó là một bài toán *gia tăng* với các biến được thêm liên tục — SLAM thời gian thực — thì GTSAM với iSAM2 có lợi thế cấu trúc.

Thị giác: [OpenCV](#) cho gần như mọi thứ ở B/07 và A/02. Một cảnh báo thực dụng: kiểm xem build của bạn có bật đúng cờ kiến trúc (SIMD) không — đây là một lỗi hiệu năng phổ biến và dễ sửa (D/20 mục 4B).

1.5 Bộ dữ liệu và công cụ đánh giá

Công cụ: [evo](#) dễ dùng; [rpg_trajectory_evaluation](#) [2] chặt chẽ hơn về căn chỉnh. Với cả hai, chọn đúng chế độ căn chỉnh theo C/18 mục 2B — mặc định không phải lúc nào cũng đúng cho cấu hình của bạn, và đây là lỗi phổ biến nhất trong đánh giá.

Bộ dữ liệu, xếp theo cái chúng dùng để đo:

Đo độ chính xác trong điều kiện tốt: EuRoC [3] (drone, trong nhà, có motion capture), TUM-VI [4] (cầm tay, có hiệu chuẩn quang trắc), TUM-RGBD [5] (RGB-D, định nghĩa gốc của ATE/RPE), KITTI [6] (xe, ngoài trời).

Đo hoạt động dài hạn và thay đổi môi trường: OpenLORIS [7] — một trong rất ít bộ chạm vào C/13 mục 4C.

Đo localization trong điều kiện thay đổi: Aachen Day-Night [8].

CẦN NHỚ

Không bộ dữ liệu công khai nào đủ, và đây là nhắc lại D/21 mục 4A vì nó là điều quan trọng nhất của mục này.

Ba lý do: chúng không chứa các chế độ hỏng ở D/19; chúng quá ngắn cho các vấn đề ở C/13; và chúng đo ATE trung bình trên các chuỗi hệ thống chạy được, trong khi sản phẩm cần tỉ lệ thất bại và hành vi ở đuôi.

Dùng chúng cho đúng việc: *so sánh thuật toán trong điều kiện kiểm soát, và phát hiện hồi quy*. Không dùng chúng để kết luận hệ thống sẵn sàng cho khách hàng.

Việc cần làm song song: xây bộ dữ liệu nội bộ theo ba phần ở D/21 mục 4A — chuỗi đại diện, chuỗi nghịch cảnh, và chuỗi từ hiện trường. Phần thứ ba có giá trị nhất và chỉ có được nhờ vòng lặp fleet analytics.

1.6 Kết nối

Chương này là tham khảo cho toàn cây, và cách dùng đúng là tra ngược: khi đọc một chương và muốn xem nó được cài đặt thế nào, quay lại Bảng 1.1 và mục 3.

Nối xuống dưới: [E/24-lo-trinh](#) giai đoạn hai là chỗ chương này được dùng nhiều nhất — đọc ba hệ thống theo thứ tự, và với mỗi hệ vẽ lại đồ thị nhân tử cùng danh sách biến nào bị fix. Đó là bài tập biến việc đọc mã từ tiêu thụ thụ động thành một việc có sản phẩm kiểm được.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao với Phần C và Phần D thì đọc mã là cách học chính chứ không phải bổ sung?
2. Nêu ba nguyên tắc đọc mã nguồn có hiệu quả, và nói nguyên tắc nào tiết kiệm nhiều thời gian nhất.
3. Với mỗi hệ trong sáu hệ ở Bảng 1.1, nói nó dạy được gì mà năm hệ kia không dạy.
4. Tiêu chí chọn giữa Ceres và GTSAM là gì?
5. Vì sao không bộ dữ liệu công khai nào đủ, và chúng nên được dùng cho việc gì?
6. Khi dùng `evo` hoặc `rpg_trajectory_evaluation`, lỗi phổ biến nhất là gì?

Nguồn

- [1] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018, mức tin B; tiền ấn nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [2] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [3] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC Micro Aerial Vehicle Datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016, mức tin A.
- [4] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2018.
- [5] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [6] A. Geiger, P. Lenz, and R. Urtasun, “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2012.
- [7] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [8] T. Sattler, W. Maddern, C. Toft, et al., “Benchmarking 6DOF Outdoor Visual Localization in Changing Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; bộ Aachen Day-Night, 2018.

Lộ trình (Roadmap)

PHỤ THUỘC Không có — chương này nên đọc trước Phần A, rồi quay lại sau mỗi giai đoạn. Nó là bản đồ chứ không phải nội dung.

TÓM TẮT

Chương này nói thứ tự nên đi, và quan trọng hơn, *lý do* cho từng thứ tự. Năm giai đoạn: nền tảng không rút ngắn được; đọc hệ thống thật; hợp nhất với hiệu chuẩn; đối mặt thực tế; và cuối cùng là nghiên cứu. Điểm khác biệt của chương so với một danh sách đọc thông thường nằm ở hai chỗ. Thứ nhất, mỗi thứ tự đều có lý do suy ra từ *cấu trúc phụ thuộc* của lĩnh vực — Lie group phải trước bundle adjustment vì BA cần Jacobian trên đa tạp; observability phải sau BA vì nó là câu hỏi về ma trận của BA. Những quan hệ ấy không đổi theo thời gian, nên chương này gần như không hết hạn. Thứ hai, và là luận điểm riêng của chương: *giai đoạn bốn — tự dựng dữ liệu và đo thất bại — bị bỏ qua nhiều nhất và có giá trị cao nhất*. Phần lớn người học nhảy từ giai đoạn ba sang giai đoạn năm, và hậu quả là họ cải tiến những thứ không phải vấn đề. Nếu chỉ nhớ một điều: *cải tiến trước khi đo là cách nhanh nhất để tối ưu sai thứ*.

2.1 Vì sao thứ tự quan trọng

Một lộ trình có thể chỉ là một danh sách sắp xếp tùy tiện. Lộ trình này thì không, và lý do đáng nói vì nó cũng là lý do chương này không hết hạn.

Lĩnh vực này có một *cấu trúc phụ thuộc* thật: một số thứ không hiểu được nếu chưa hiểu thứ khác, và quan hệ ấy là quan hệ logic chứ không phải quy ước sự phạm.

Ba ví dụ cụ thể từ chính cây này. *Lie group phải đến trước bundle adjustment*, vì BA cần Jacobian của phần dư theo tư thế, và tư thế sống trên đa tạp — không có A/03 thì công thức ở A/04 là hộp đen. *Observability phải đến sau BA*, vì toàn bộ A/05 là một câu hỏi về ma trận thông tin mà A/04 dựng lên. *Hiệu chuẩn phải đến sau cả hai*, vì C/14 chỉ có nghĩa khi đã có khung MAP để thả biến vào và đã có lý thuyết kích thích để biết khi nào thả được.

Những quan hệ ấy không phụ thuộc vào công nghệ hiện tại. Chúng vẫn đúng nếu mọi framework trong E/23 biến mất.

Có một nguyên tắc thứ hai chi phối lộ trình, ít hiển nhiên hơn: *học một thứ trước khi cần nó thì quên; học nó đúng lúc cần thì nhớ*. Đây là lý do lộ trình xen kẽ lý thuyết với thực hành thay vì học hết lý thuyết rồi mới làm — và là lý do mỗi chương trong cây có một mini project.

2.2 Giai đoạn 1 — Nền tảng, không rút ngắn được

Nội dung: A/01 (hình học đa thị) → A/02 (mô hình camera) → A/03 (Lie group) → A/04 (MAP và BA) → A/05 (observability) → A/06 (bất định).

Vì sao thứ tự này: đã giải thích ở mục 1 cho ba quan hệ chính. Thêm hai quan hệ: chương 1 trước chương 2 vì chương 2 lắp vào chỗ chương 1 bỏ trống; chương 6 cuối cùng vì nó cần cả Λ từ chương 4 lẫn điều kiện khả nghịch từ chương 5.

Vì sao không rút ngắn được: đây là phần duy nhất trong cây mà tôi khẳng định như vậy, nên nó đáng biện minh. Mọi thứ ở bốn phần sau đều là các cách khác nhau để dùng sáu chương này. Một người bỏ qua A/03 sẽ đọc mọi Jacobian như bùa chú; một người bỏ qua A/05 sẽ chẩn đoán sai mọi lần hệ thống hỏng. Không có đường tắt, và cố đi tắt sẽ tốn nhiều thời gian hơn.

Cách đo tiến độ, và đây là điểm quan trọng: *tự dẫn lại được trên giấy*. Cụ thể: dẫn lại ràng buộc epipolar từ tính đồng phẳng; dẫn lại $\partial(R\mathbf{p})/\partial\phi$; dẫn lại phần bù Schur; dẫn lại bảy bậc tự do của SfM mono. Nếu chưa dẫn lại được thì chưa xong, bất kể đã đọc bao nhiêu lần.

Mini project nên làm: chương 1 (pipeline hai khung và ba cấu hình suy biến), chương 3 (kiểm mọi Jacobian bằng sai phân số — bài này tạo ra bộ unit test dùng suốt về sau), và chương 5 (phổ giá trị riêng trên bốn quỹ đạo).

Thời gian: đây là phần dài nhất và nên là phần dài nhất.

2.3 Giai đoạn 2 – Đọc hệ thống thật

Nội dung: đọc B/07 tới B/12 song song với việc đọc mã nguồn ba hệ thống, theo thứ tự ở E/23 mục 2: VINS-Fusion, rồi OpenVINS, rồi ORB-SLAM3.

Vì sao thứ tự hệ thống này: VINS-Fusion ngắn nhất và cấu trúc rõ nhất, nên nó là chỗ học cách đọc mã SLAM; OpenVINS cho thể giới bộ lọc và — quan trọng hơn — cho một bộ mô phỏng có chân trị, thứ cần cho C/18; ORB-SLAM3 đầy đủ nhất và phức tạp nhất, nên đọc nó cuối.

Bài tập bắt buộc cho mỗi hệ, và đây là phần biến việc đọc mã từ tiêu thụ thụ động thành một việc có sản phẩm: *vẽ lại đồ thị nhân tử và liệt kê chính xác biến nào được fix*. Với mỗi hệ, trả lời: những biến nào trong vector trạng thái? Những loại nhân tử nào? Cái gì bị fix và vì sao? Nó xử lý gauge thế nào?

Bài tập ấy có giá trị vượt xa việc hiểu ba hệ thống: nó buộc bạn đặt mỗi hệ vào Bảng ở 00-map, và sau ba lần bạn sẽ thấy chúng là ba ô của cùng một bảng.

Cách đo tiến độ: bạn giải thích được vì sao mỗi hệ đưa ra các lựa chọn kiến trúc của nó, và bạn chỉ ra được ít nhất một chỗ mà mã khác với bài báo.

2.4 Giai đoạn 3 – Hợp nhất với hiệu chuẩn

Nội dung: C/14 cùng mini project của nó, cộng C/13 và C/15.

Vì sao giai đoạn này đứng riêng: vì nó là chỗ luận đề trung tâm của cây được chứng minh bằng tay. Mini project ở C/14 — viết một bộ giải rồi chạy bốn bài toán khác nhau chỉ bằng cách đổi danh sách biến — là bài tập làm sáng tỏ luận đề rõ nhất trong cả cây.

Quy trình đề nghị: chạy Kalibr trên dữ liệu của mình để có cảm giác; rồi tự viết một bài toán hiệu chuẩn bằng Ceres hoặc GTSAM; rồi *thêm dần biến vào* cho tới khi nó trở thành VIO. Mỗi lần thêm một biến, hỏi: biến này cần chuyển động gì để quan sát được? Đó là Bảng kích thích ở A/05 được kiểm bằng tay.

Cách đo tiến độ: bạn có một bộ mã chạy được cả bốn cấu hình, và bạn giải thích được phổ giá trị riêng của từng cấu hình.

Vì sao C/13 thuộc giai đoạn này: vì quản lý bản đồ và hiệu chuẩn chia nhau một câu hỏi — cái gì được coi là cố định và cái gì được phép đổi. Và vì chương 13 là chương bị coi nhẹ nhất, nên đặt nó cạnh một chương mà người học đang hào hứng là cách để nó không bị bỏ.

2.5 Giai đoạn 4 – Đối mặt thực tế

Nội dung: C/18 (phương pháp đánh giá) và D/19 (chế độ hồng), rồi tự dựng dữ liệu trong môi trường mục tiêu, đo thất bại, phân loại nguyên nhân.

Đây là giai đoạn mà tôi cho là quan trọng nhất và bị bỏ qua nhiều nhất, nên nó đáng một lập luận đầy đủ.

A. Vì sao nó bị bỏ qua

Ba lý do, và cả ba đều dễ hiểu.

Một — nó không có cảm giác tiến bộ. Đọc một bài báo mới cho cảm giác học được điều gì đó; thu ba giờ dữ liệu trong hành lang văn phòng thì không.

Hai — nó không có sản phẩm khoe được. Không có bảng số nào để đưa vào báo cáo, không có thuật toán mới nào.

Ba — nó đòi hỏi thừa nhận rằng mình chưa biết vấn đề là gì. Sau ba giai đoạn học, cảm giác tự nhiên là bắt tay vào cải tiến. Việc dừng lại để đo là một sự thừa nhận rằng ta chưa biết cần cải tiến cái gì.

B. Vì sao nó quan trọng nhất

Lập luận gọn: *cải tiến trước khi đo là cách nhanh nhất để tối ưu sai thứ.*

Cụ thể hơn. Sau ba giai đoạn, bạn có một danh sách các thứ có thể cải tiến — front-end tốt hơn, back-end tốt hơn, bản đồ tốt hơn. Danh sách ấy đến từ việc đọc, nên nó phản ánh *những gì cộng đồng đang quan tâm*, không phản ánh *vấn đề của bạn*.

Trong thực tế, khi đo, người ta thường phát hiện rằng vấn đề thật là một thứ không có trong danh sách: một chỗ trong môi trường có gương; ánh sáng thay đổi vào một giờ nhất định; bánh xe trượt trên một loại sàn cụ thể; hiệu chuẩn trôi sau hai giờ chạy. Không vấn đề nào trong số đó được giải bằng một detector tốt hơn.

Theo C/18 mục 5D: *SOTA thật nằm ở đuôi phân bố.* Và đuôi phân bố của *bạn* chỉ đo được trên dữ liệu của bạn.

C. Làm gì cụ thể

Bước một — dựng bộ dữ liệu, theo ba phần ở D/21 mục 4A: chuỗi đại diện (dài, môi trường điển hình), chuỗi nghịch cảnh (theo mini project D/19), và — nếu có thiết bị chạy thật — chuỗi từ hiện trường.

Bước hai — đo theo bốn câu hỏi ở C/18 khối Cần nhớ: chính xác tới đâu, có thành thật không, hồng thế nào và bao nhiêu lần, ở đuôi thì sao.

Bước ba — phân loại nguyên nhân, không phân loại triệu chứng. Mười lần mất bám có thể có ba nguyên nhân, và việc gộp chúng theo triệu chứng che mất điều đó. Dùng các đại lượng ở D/19 khối Cần nhớ để phân cụm.

Bước bốn — chỉ bảy giờ mới bắt đầu cải tiến, và bắt đầu từ cụm lớn nhất.

Cách đo tiến độ: bạn có một bảng xếp hạng nguyên nhân thất bại trong môi trường của bạn, với tần suất. Nếu bảng ấy trùng với danh sách bạn đoán trước khi đo thì hoặc bạn rất giỏi hoặc bạn chưa đo đủ khó.

2.6 Giai đoạn 5 – Nghiên cứu

Nội dung: D/22 để chọn ứng viên, rồi `research-rules.md` để làm đúng cách.

Quy trình, và thứ tự quan trọng.

Một — chọn một hướng, từ D/22 hoặc — tốt hơn — từ bảng xếp hạng nguyên nhân ở giai đoạn 4. Hướng đến từ vấn đề thật của bạn có ba lợi thế: bạn có dữ liệu, bạn có tiêu chí thành công, và bạn biết nó đáng giải.

Hai — chạy phép kiểm của hướng đó (D/22 khối *Cần nhớ*). Vài ngày, và nó có thể tiết kiệm một năm.

Ba — nếu phép kiểm xác nhận, nâng lên thành khảo sát đúng chuẩn: `research-rules.md` bước 0 tới bước 4. Đọc 20–30 công trình gần nhất của riêng hướng đó, có nhật ký tìm kiếm, có ma trận tổng hợp.

Bốn — tái hiện hai tới ba baseline trước khi đề xuất bất cứ gì. Theo C/18 mục 5B: nếu baseline của bạn không đạt con số công bố thì mọi so sánh về sau vô nghĩa. Và bước này thường dạy nhiều hơn cả bước năm.

Năm — chỉ bây giờ mới đề xuất.

Cách đo tiến độ: bạn có một `00-question.md` theo `research-rules.md` mục 7 — một câu hỏi đủ cụ thể để có đáp án đúng/sai, một giả thuyết kèm mức tin bằng số, và một tiêu chí falsify viết trước khi chạy.

CẦN NHỚ

Lộ trình rút gọn, nếu bạn chỉ có thời gian cho phần xương sống.

Bảy chương phải đọc kỹ và tự dẫn lại được: A/03 (Lie group), A/04 (MAP và BA), A/05 (observability), B/07 (front-end), B/09 (paradigm back-end), C/13 (quản lý bản đồ), C/14 (hiệu chuẩn).

Ba mini project có tỉ lệ giá trị trên thời gian cao nhất: kiểm Jacobian bằng sai phân số (A/03 — tạo ra bộ test dùng suốt); phổ giá trị riêng trên bốn quỹ đạo (A/05 — làm observability thành thứ nhìn thấy được); và đi từ hiệu chuẩn tới SLAM bằng cách thả biến (C/14 — chứng minh luận đề trung tâm).

Một hệ thống đọc trọn vẹn: VINS-Fusion, vì nó ngắn nhất mà vẫn đủ các thành phần.

Một việc không được bỏ: giai đoạn 4. Kể cả khi rút gọn mọi thứ khác, hãy tự thu dữ liệu trong môi trường của mình và đo thất bại. Đây là việc phân biệt một người hiểu lĩnh vực với một người hiểu *bài toán của mình* — và chỉ có cái thứ hai mới tạo ra thứ dùng được.

2.7 Một gợi ý về chỗ bắt đầu

Một nhận xét cuối, và nó là một nhận xét về phương pháp chứ không về nội dung.

Nếu bạn đang có một bài toán ứng dụng cụ thể — ví dụ docking bằng marker với yêu cầu độ chính xác milimet, chủ đề của `../marker/` — thì bài toán ấy thường là chỗ tốt hơn để bắt đầu so với việc đi tuần tự qua cây này.

Lý do: một bài toán ứng dụng hẹp chứa *bản thu nhỏ* của gần như toàn bộ lộ trình. Nó có hiệu chuẩn (C/14); nó có observability và lưỡng nghĩa (A/05); nó có ngân sách sai số (A/06); nó có yêu cầu sản xuất (D/21); và nó có một tiêu chí thành công rõ ràng mà không benchmark nào cho bạn.

Quan trọng hơn cả: nó cho bạn *lý do để cần* mỗi chương. Học A/05 vì tò mò thì khó nhớ; học nó vì tư thế của bạn đang lưỡng nghĩa và bạn cần biết vì sao thì nhớ rất lâu. Đây là nguyên tắc “học đúng lúc cần” ở mục 1, áp dụng ở quy mô lớn nhất.

Cách kết hợp đề nghị: dùng bài toán ứng dụng làm *trực*, và dùng cây này làm *tài liệu tra* — mỗi lần gặp một chỗ không hiểu, tìm chương tương ứng và đọc nó trọn vẹn. Sau vài tháng bạn sẽ đã đọc phần lớn cây, và mỗi chương đều gắn với một vấn đề thật bạn từng gặp.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba quan hệ phụ thuộc trong giai đoạn 1 và lý do logic của mỗi quan hệ.
2. Cách đo tiến độ của giai đoạn 1 là gì, và vì sao “đã đọc xong” không phải một cách đo?
3. Bài tập bắt buộc khi đọc mỗi hệ thống ở giai đoạn 2 là gì, và nó cho bạn thấy điều gì sau ba lần?
4. Nêu ba lý do vì sao giai đoạn 4 bị bỏ qua, và lập luận vì sao nó quan trọng nhất.
5. Vì sao một hướng nghiên cứu đến từ bảng xếp hạng nguyên nhân của bạn tốt hơn một hướng chọn từ danh sách?
6. Trong quy trình năm bước của giai đoạn 5, bước nào thường dạy nhiều nhất và vì sao?
7. Vì sao một bài toán ứng dụng cụ thể thường là chỗ tốt hơn để bắt đầu so với đi tuần tự qua cây?

Nguồn

- [1] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018, mức tin B; tiền ấn nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [2] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [3] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC Micro Aerial Vehicle Datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016, mức tin A.
- [4] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2018.
- [5] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [6] A. Geiger, P. Lenz, and R. Urtasun, “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2012.
- [7] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [8] T. Sattler, W. Maddern, C. Toft, et al., “Benchmarking 6DOF Outdoor Visual Localization in Changing Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; bộ Aachen Day-Night, 2018.

Nguồn

- [1] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018, mức tin B; tiền ấn nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [2] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [3] M. Burri, J. Nikolic, P. Gohl, et al., “The EuRoC Micro Aerial Vehicle Datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016, mức tin A.
- [4] D. Schubert, T. Goll, N. Demmel, et al., “The TUM VI Benchmark for Evaluating Visual-Inertial Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2018.
- [5] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [6] A. Geiger, P. Lenz, and R. Urtasun, “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2012.
- [7] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [8] T. Sattler, W. Maddern, C. Toft, et al., “Benchmarking 6DOF Outdoor Visual Localization in Changing Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; bộ Aachen Day-Night, 2018.